Enlear Academy · Follow publication

Data Encryption on AWS — Part 02

5 min read · Jan 18, 2020

Manoj Fernando Follow

Listen Share

Photo by Dominik Scythe on Unsplash

In part 01, we discussed the main concepts around AWS KMS. You can find the Youtube video related to this blog post here.

OpenSSL and **AWS Encryption SDK** are used for Client-Side Encryption outside AWS. This blog post is focused on how to interact with KMS using AWS CLI and OpenSSL for data encryption and decryption. In the next part, we will also discuss the AWS Encryption SDK with examples.

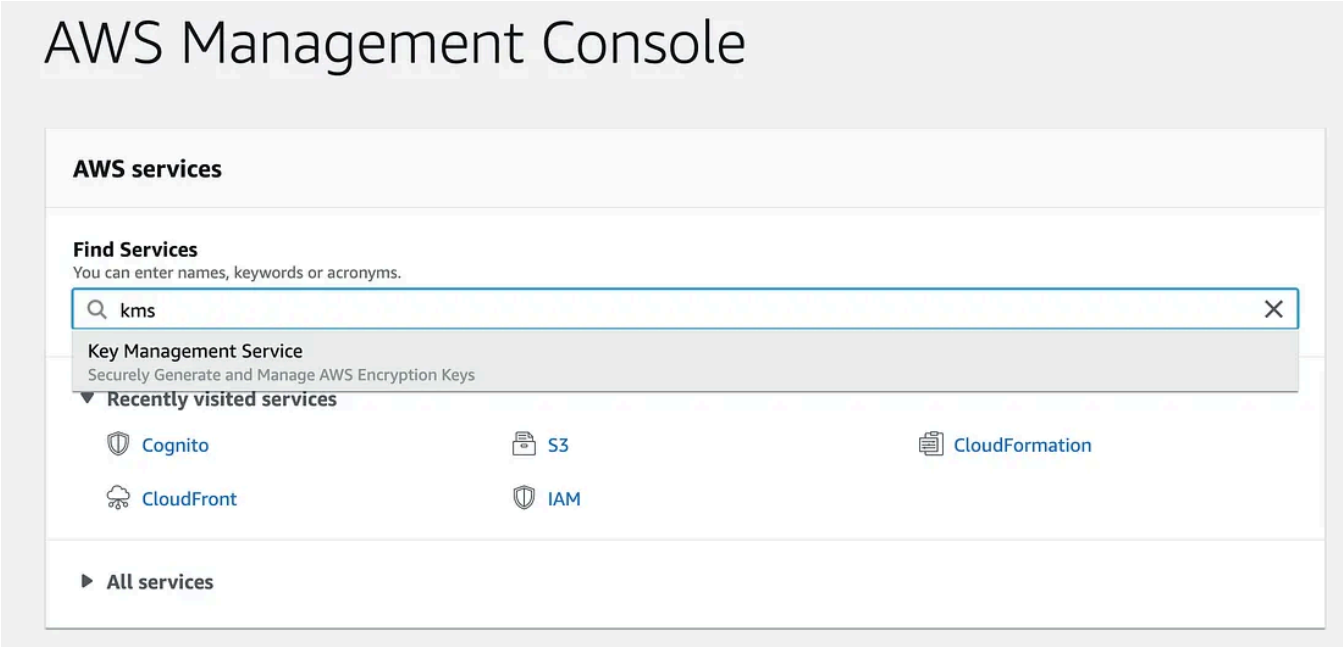
Encrypt/Decrypt using OpenSSL

OpenSSL is a full-featured cryptographic library that we can use to communicate with AWS KMS over the command-line interface. (You can install the OpenSSL toolkit for your operating system)

Step 01 — Creating a CMK

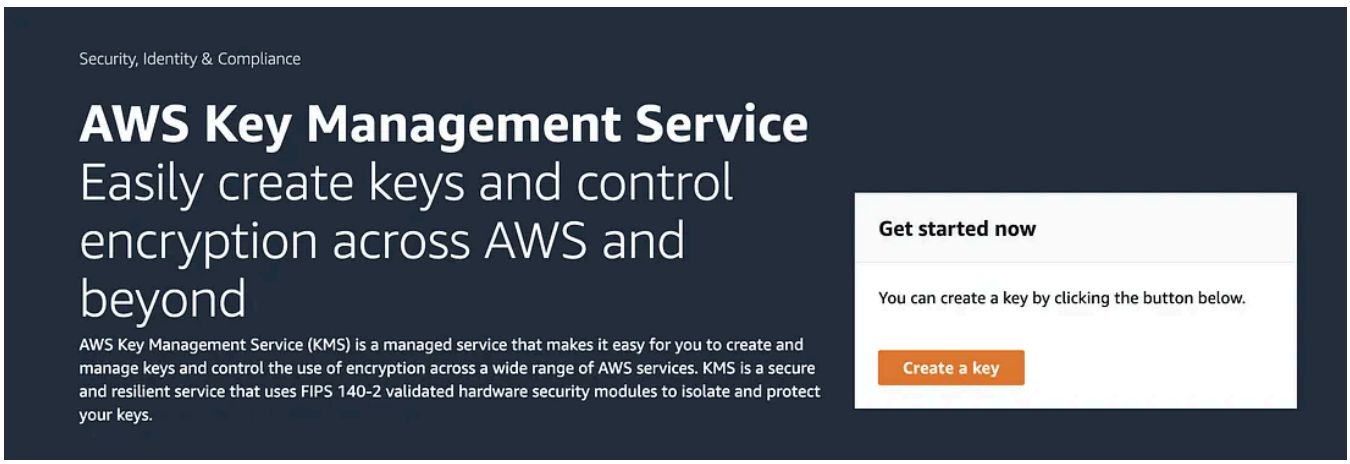
Let's start by creating a CMK in our AWS account. This can be done using the AWS Console, AWS SDKs or AWS CLI. I use the AWS Console.

Login to your AWS account and go to AWS KMS.



Select KMS and open the KMS console

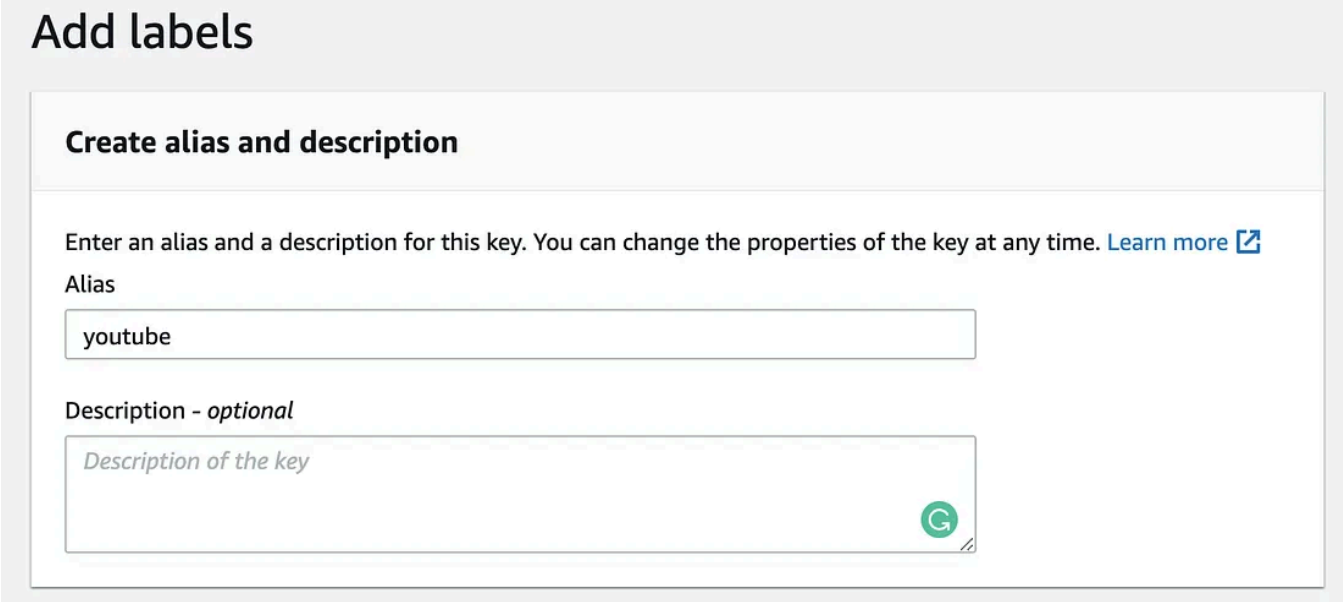
Select the region N.Virginia (us-east-1) from the top right side of the console and click "Create Key"



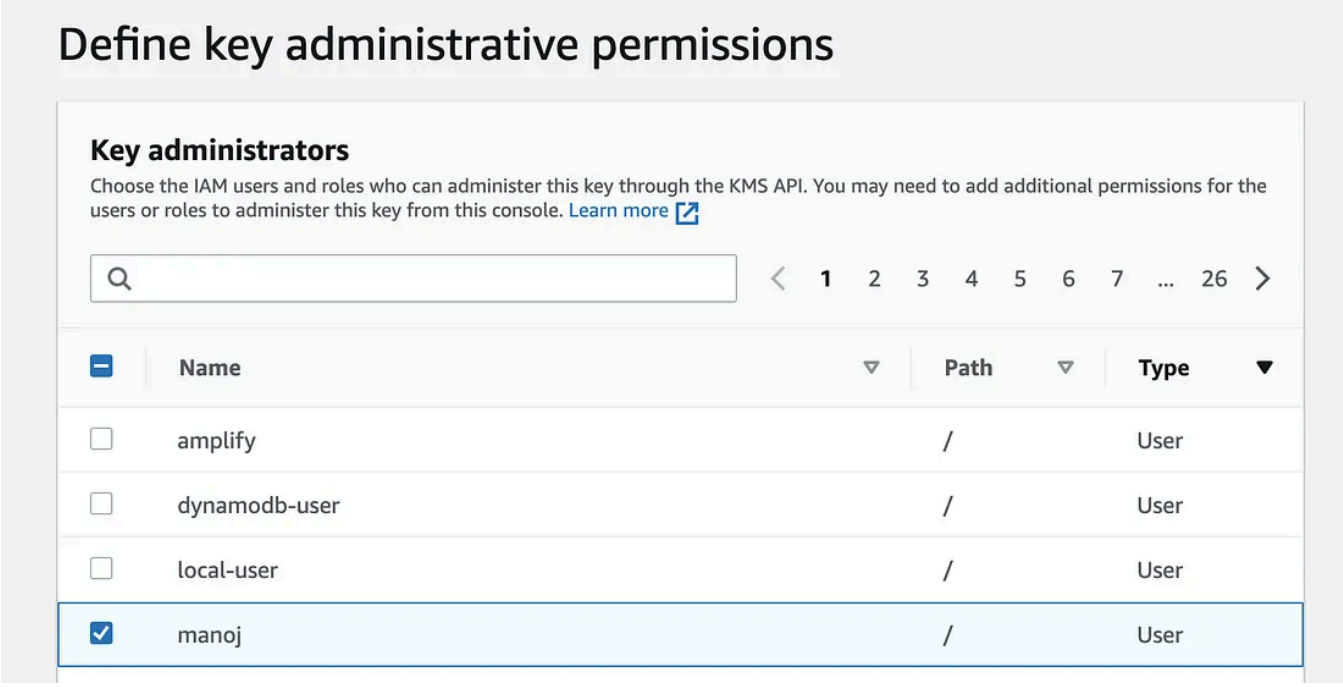
Select Symmetric encryption type and click “Next”. In Symmetric Encryption, the same key is used for both encryption and decryption. AWS recommends using Symmetric CMK for most cases.

> *“Use a symmetric CMK for most use cases that require encrypting and decrypting data. The symmetric encryption algorithm that AWS KMS uses is fast, efficient, and assures the confidentiality and authenticity of data.” — AWS Documentation*

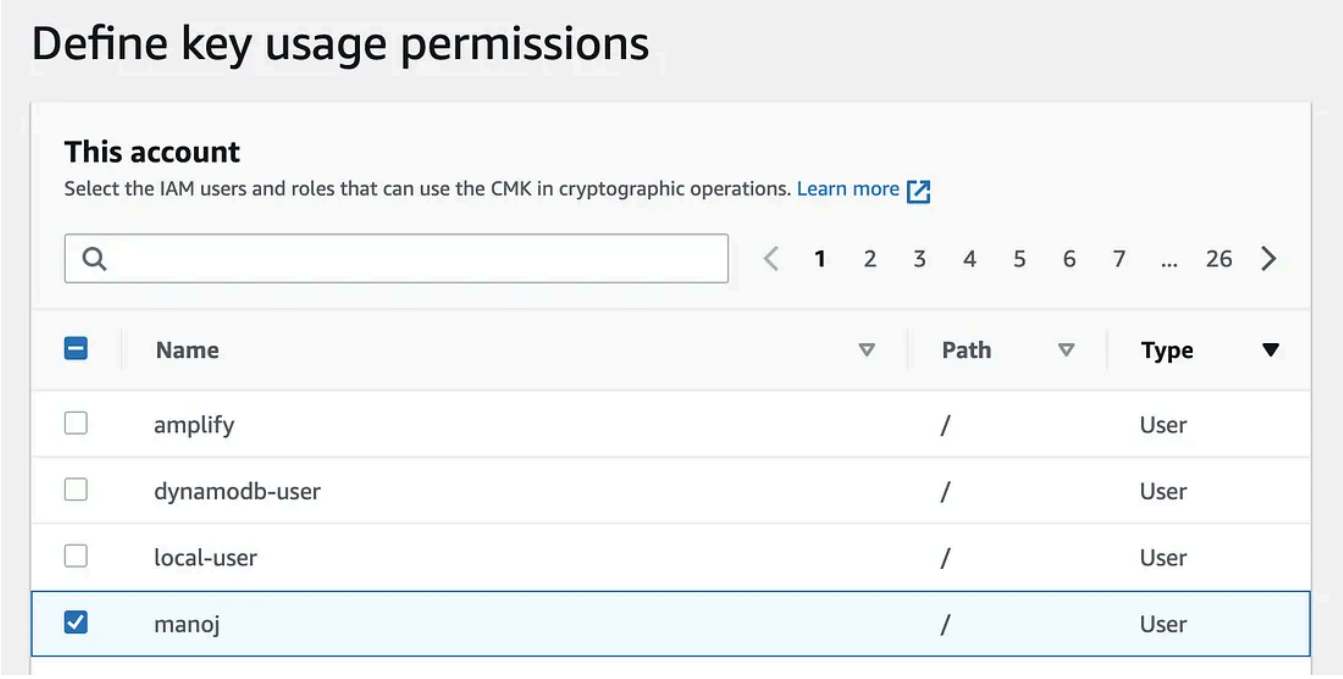
Provide an alias to the key in the next step. Alias is useful to reference the CMK easily.



Tell KMS about the key administrators. By default, the root user has all the permissions. You can select IAM users who can administrate the key and use the key. Click next.



Now select the IAM users who need key usage permissions.



Finally, review the permissions and click finish to create the CMK.

Step 02 — Generating Data-Keys for the CMK

A CMK only allows encrypting data that is less than 4KBs. If we have a large payload to encrypt, then we need Data Keys generated from that CMK. (See the video for more details).

Let's use AWS CLI to call KMS service and generate data keys for the CMK we just created.

Note: Follow the instruction to install AWS CLI and configure in your operating system.

We refer to the CMK by the alias (e.g. youtube) we have provided during the creation process at step 01.

```
aws kms generate-data-key --key-id alias/youtube --key-spec AES_256
--region us-east-1
```

Response (This is mock data)

```
{"KeyId": "arn:aws:kms:us-east-1:123456789:key/bbee76a1-bd25-4d57-
81d8-38ff2b26468a",

"Plaintext": "7DmPVPgzJ8exc9+AekcEmVL7jdv0RWMxPgA4JlrpE4k=",

"CiphertextBlob":
"ADIDAHiiF6PCTM1Hou+61r+M/pyUfwSizO02mH9+pIa0gaFRWwFF+FoN25Pm+tdPZiB
0paGRAAAAfjB8BgkqhkiG9w0BBwabbzBtAgEAMGgGCSqGSIb3DQEHATAeBglghkgBZQM
EAS4wEQQMIB9YpWJsDdZjP4BVAgEQgDvigjj2IaJoDmXJPS2AWG6OHqMwI8H5ybsS6l0
Rt26fVUskQTxxWvCzkLSqssqi3bDnEysfaxN/ryXO7w=="}
```

It returns both the Plaintext version of the data key and the Encrypted or Ciphertext version of the same data key. Both these keys are base64 encoded. So let's decode and save them into **datakey** and **encrypted-datakey** files

```
echo "7DmPVPgzJ8exc9+AekcEmVL7jdv0RWMxPgA4JlrpE4k=" | base64
--decode > datakey

echo
"ADIDAHiiF6PCTM1Hou+61r+M/pyUfwSizO02mH9+pIa0gaFRWwFF+FoN25Pm+tdPZiB
0paGRAAAAfjB8BgkqhkiG9w0BBwabbzBtAgEAMGgGCSqGSIb3DQEHATAeBglghkgBZQM
```

```
EAS4wEQQMIB9YpWJsDdZjP4BVAgEQgDvigjj2IaJoDmXJPS2AWG6OHqMwI8H5ybsS6l0
Rt26fVUskQTxxWvCzkLSqssqi3bDnEysfaxN/ryXO7w=="
| base64 --decode > encrypted-datakey
```

We will use them in the next step.

Step 03 — Encrypting data with Plaintext Data-Key

Now we use the Plaintext data key to encrypt our data.

First of all, we need data to encrypt. Let's create **password.txt** file with some data. In general, this will be the sensitive data that we need to protect.

```
echo "My database password" > password.txt
```

Now, let's use the **datakey** to encrypt our sensitive data. We will output the encoded data into a file called **secret.txt**.

```
openssl enc -in ./passwords.txt -out ./passwords-encrypted.txt -e -
aes256 -k fileb://./datakey
```

After encrypting the data, we must NOT forget to delete the plaintext-datakey. Otherwise, anyone can use that key to decrypt our secret data.

```
rm datakey
```

Step 04 — Decrypting data with Encrypted Data Key

For that, we use encrypted-data-key that was stored with the encrypted data. We already discussed KMS concepts in-depth in the previous blog post as well as in the Data Encryption on AWS video.

We need to pass the encrypted-data key to KMS and request for the plaintext-data key. It will return the same plain text data key that we used to encrypt the sensitive data.

```
aws kms decrypt --ciphertext-blob fileb://./encrypted-datakey  --
region us-east-1
```

```
[Output - Mock data]
{
"Plaintext": "xyQtd+/oB0ob1Gr9dmkQ4JBSR1+jQRZrK1sLAVdJIHg=",
"KeyId": "arn:aws:kms:us-east-1:123456789:key/beae46a1-bd25-4d37-
81d8-38ff1b26469a"
}
```

Great! Now we can use this Plaintext data key to decrypt our data. But first, let's do base64 decode and save it as **datakey** again.

```
echo "xyQtd+/oB0ob1Gr9dmkQ4JBSR1+jQRZrK1sLAVdJIHg=" | base64
--decode > datakey
```

Now finally we can decrypt our encrypted data with the **datakey** that we received. The decrypted sensitive data is output to a file called passwords-decrypted.txt.

```
openssl enc -in ./passwords-encrypted.txt -out ./passwords-
decrypted.txt -d -aes256 -k fileb://./datakey
```

Now if you open the **passwords-decrypted.txt** you should find the original plaintext data.

Congratulations! We have successfully completed the encryption and decryption of our sensitive data.

AWS Security Encryption

Follow

Published in Enlear Academy